

Report Εργασίας 1 ΨΕΕ

PART 1: Helper function that it was introduced

To begin with, a helper function *get_frequency_grid* was introduced to optimize the codebase and eliminate redundancy (DRY principle), as permitted by the assignment guidelines. It efficiently handles the generation of the 2D frequency meshgrid, computes the centered spatial frequency coordinates $\bar{u}_1$ and $\bar{u}_2$, and calculates the Euclidean distance D from the spectral center. Abstracting this repetitive geometric logic into a standalone utility function significantly streamlined the implementation of all filter design functions (designLowFilter, designHighFilter, designDirectionalFilter, designSmoothDirectionalFilter), improving overall code readability and robustness.

Python Implementation of *get_frequency_grid()*:

```
def get_frequency_grid(N1: int, N2: int):  
  
    u1 = np.arange(N1)  
    u2 = np.arange(N2)  
    U1, U2 = np.meshgrid(u1, u2, indexing='ij')  
  
    #centered coordinates(shifted indices)  
    u1_bar = U1 - N1/2  
    u2_bar = U2 - N2/2  
  
    #calculates Euclidean distance from the spectral center  
    D = np.sqrt(u1_bar**2 + u2_bar**2)  
  
    return u1_bar, u2_bar, D
```

PART 2: Functions

- 1. Implementation of 2D Discrete Fourier Transform(DFT2 function):**The frequency domain representation of the input image was computed using a custom DFT2 function, which utilizes the Fast Fourier Transform (FFT) algorithm for computational efficiency. The implementation includes a pre-processing stage that ensures even spatial dimensions by trimming the input array where necessary. To facilitate spectral analysis, an optional centering mechanism was incorporated; by modulating the spatial signal with a $(-1)^{(n_1+n_2)}$ factor, the zero-frequency (DC) component is shifted to the center of the frequency plane. This approach bypasses the need for post-transform circular shifts, resulting in a complex-valued spectrum $F(k_1, k_2)$ ready for subsequent filtering or analysis.

Python Implementation of DFT2():

```
def DFT2(f: np.ndarray, N1: int, N2: int, fftshift: int) -> np.ndarray:  
  
    # Create a copy to avoid accidentally modifying the original image  
    img = f.copy()  
  
    # Dimension Check: If N1 or N2 are not even, remove the last row/column  
    if N1 % 2 != 0:  
        N1 -= 1  
        img = img[:N1, :]  
  
    if N2 % 2 != 0:
```

```

N2 -= 1
img = img[:, :N2]

# Centering trick: Move the DC component to the center if fftshift == 1
if fftshift == 1:
    # Create grids for the indices n1 and n2
    n1_indices, n2_indices = np.indices((N1, N2))

    # Multiply each pixel by (-1)^(n1+n2)
    # Utilizes the property (-1)^(n1+n2) = 1 if (n1+n2) is even, -1 if odd
    multiplier = (-1) ** (n1_indices + n2_indices)
    img = img * multiplier

# Compute the 2D DFT using numpy's optimized FFT implementation
F = np.fft.fft2(img)

return F

```

2. **2D Inverse Discrete Fourier Transform Implementation (iDFT2 function):** The reconstruction of the image from its frequency representation was achieved through a custom iDFT2 function using the Inverse Fast Fourier Transform (IFFT). To ensure a valid spatial domain output, the function extracts the real component of the result, effectively neutralizing minor imaginary residuals caused by numerical floating-point errors. Furthermore, the implementation accounts for previous spectral centering by reapplying the spatial modulation factor $(-1)^{(n1+n2)}$ when the fftshift flag is active. This process successfully restores the image to its original spatial orientation and intensity scale, maintaining consistency with the forward transform parameters.

Note: Alternatively, the iDFT2 could have been implemented using the DFT2 function by leveraging the complex conjugate property; however, `np.fft.ifft2` was preferred to maintain code simplicity and readability.

Python Implementation of iDFT2():

```

def iDFT2(F: np.ndarray, N1: int, N2: int, fftshift: int) -> np.ndarray:

    f_complex = np.fft.ifft2(F) #Computes inverse 2D DFT using numpy's optimized IFFT

    f_real = np.real(f_complex) #Extracts real part to remove negligible imaginary components from
    floating-point errors

    #Reverses the centering modulation if fftshift was originally applied
    if fftshift == 1:
        n1_indices, n2_indices = np.indices((N1, N2))
        multiplier = (-1) ** (n1_indices + n2_indices)
        f_real = f_real * multiplier

    return f_real

```

3. **Frequency Domain Low-Pass Filter Design (designLowFilter function):** To perform image smoothing and noise reduction, a versatile designLowFilter function was implemented, supporting Ideal, Butterworth, and Gaussian transfer functions. The filters are constructed on a 2D frequency grid by calculating the Euclidean distance of each component from the spectral center. The Ideal filter provides a binary sharp cutoff, while the Butterworth and Gaussian variants offer smoother transitions to mitigate the ringing artifacts (Gibbs phenomenon) typically associated with abrupt frequency truncations. By adjusting the cutoff frequency D_0 and the filter order n ,

the frequency response is precisely tuned to suppress high-frequency components while preserving the essential structural information of the image.

Python Implementation of designLowFilter():

```
def designLowFilter(N1: int, N2: int, param: tuple) -> np.ndarray:

    filter_type, D0, n = param
    , , D = get_frequency_grid(N1, N2)

    if filter_type == 0: # Ideal
        H = (D <= D0).astype(float)
    elif filter_type == 1: # Butterworth
        H = 1.0 / (1.0 + (D / (D0 if D0 != 0 else 1e-8))**(2 * n))
    elif filter_type == 2: # Gaussian
        H = np.exp(-(D**2) / (2 * (D0**2)))
    else:
        raise ValueError("Invalid filter type. Select 0, 1, or 2.")
    return H
```

4. **Frequency Domain High-Pass Filter Design (designHighFilter function):** The high-pass filtering stage was implemented by leveraging the mathematical relationship between complementary filters. Rather than redundantly defining new transfer functions for Ideal, Butterworth, and Gaussian high-pass filters, the designHighFilter function follows a modular programming approach by invoking the previously defined designLowFilter and calculating its complement, $H_{hp}(u, v) = 1 - H_{lp}(u, v)$. This strategy eliminates code duplication, reduces the risk of implementation errors, and ensures that the high-frequency boost characteristics perfectly mirror the low-pass attenuation curves. This method effectively suppresses low-frequency components while preserving or enhancing edge information and fine details within the image.

Python Implementation of designHighFilter():

```
def designHighFilter(N1: int, N2: int, param: tuple) -> np.ndarray:

    return 1.0 - designLowFilter(N1, N2, param)
```

5. **The freqFilter function:** The freqFilter function implements the fundamental frequency domain filtering process, as outlined in Section 1.1 of the assignment. Initially, the input image is transformed into the frequency domain using the DFT2 function, with the fftshift=1 flag enabled to correctly center the DC component. The filter mask H is then applied via element-wise multiplication with the frequency matrix F, directly satisfying Equation (3): $G(u_1, u_2) = H(u_1, u_2) * F(u_1, u_2)$. Finally, the filtered matrix G is transformed back to the spatial domain by calling the iDFT2 function (again with fftshift=1 to reverse the centering transformation), yielding the final output image g.

Python Implementation of freqFilter():

```
def freqFilter(f: np.ndarray, H: np.ndarray, N1: int, N2: int) -> np.ndarray:

    #Transforms the spatial image to the frequency domain.
```

```

    F = DFT2(f, N1, N2, fftshift=1) # We use fftshift=1 to ensure the DC component is at the
spectral center,
    # matching the coordinate system used during filter design.

    #Applies the filter H via element-wise (Hadamard) multiplication.
    G = F * H #Both F and H MUST have the same dimensions (N1 x N2).

    #Transforms the filtered spectrum back to the spatial domain.
    g = iDFT2(G, N1, N2, fftshift=1) #We maintain fftshift=1 to correctly reverse the spatial
modulation.

    return g

```

6. **Directional Filter (designDirectionalFilter function):** During the implementation of the designDirectionalFilter function, a mathematical discrepancy was identified in the assignment description. Specifically, the text in Section 1.4 states that the passband of the directional filters is restricted to angles within the range $[\theta_i - (\pi/2q), \theta_i + (\pi/2q)]$. However, Equation (11) dictates the condition $|\arctan(u_2/u_1) - \theta_i| \leq (\pi/q)$. If Equation (11) is strictly followed, the angular width of each filter effectively doubles to $(2\pi/q)$, which leads to significant spatial overlapping between adjacent directional filters. To satisfy the core requirement that the filter bank must be strictly "complementary" to the low-pass filter without intersecting passbands, the implementation was adapted to use the condition $(\pi/2q)$. This ensures that the entire frequency plane is perfectly partitioned into q distinct, non-overlapping slices as originally intended by the textual description.

Python Implementation of designDirectionalFilter():

```

def designDirectionalFilter(N1: int, N2: int, D0: float, q: int) -> np.ndarray:

    u1_bar, u2_bar, D = get_frequency_grid(N1, N2) # Retrieves centered coordinates and radial
distance from the helper

    H = np.zeros((N1, N2, q + 1), dtype=float) #Initializes the 3D filter bank array (N1 x N2 x
q+1)

    #H0: Base Ideal Low-Pass Filter component with cutoff D0
    H[:, :, 0] = (D <= D0).astype(float)

    theta_map = np.arctan2(u2_bar, u1_bar) % np.pi #Computes the angular map and normalize to the
range [0, pi)

    #Generates q directional sectors
    for i in range(1, q + 1):
        theta_i = (i - 1) * (np.pi / q) #Calculates the central angle for the i-th directional
filter

        #Calculates the shortest angular distance between the map and theta_i
        diff = np.abs(theta_map - theta_i)
        diff = np.minimum(diff, np.pi - diff) #Handles periodicity of directions

        condition = (diff <= (np.pi / (2 * q))) & (D > D0) #Defines the sector: must be within the
angular threshold and outside D0

        #Assigns the binary mask to the corresponding bank index
        H[:, :, i] = condition.astype(float)

    return H

```

7. **The designSmoothDirectionalFilter function:** The designSmoothDirectionalFilter function implements Equation (12) of the assignment to generate a bank of smooth directional filters. Following DRY (Don't Repeat Yourself) principles for code optimization, the function initially calls designDirectionalFilter to obtain the strict filter masks that already satisfy the magnitude constraint $\sqrt{(\overline{u1})^2 + \overline{u2}^2} > D_0$ and the angular bounds. It then utilizes the helper method get_frequency_grid to extract the centered coordinates $\overline{u1}$, $\overline{u2}$. For each directional filter slice i in $[1, q]$, the shortest angular distance from the central angle θ_i is computed, properly accounting for the angular circularity within the $[0, \pi]$ range. Finally, the smoothing factor is applied by multiplying the pre-existing mask with $\cos^k(.)$, allowing for a smooth attenuation roll-off without altering the already zeroed-out frequencies.

Python Implementation of designSmoothDirectionalFilter():

```
def designSmoothDirectionalFilter(N1: int, N2: int, D0: float, q: int, K: int) -> np.ndarray:
    #Reuse binary directional masks
    H = designDirectionalFilter(N1, N2, D0, q)

    u1_bar, u2_bar, _ = get_frequency_grid(N1, N2) #we use underscore (_) for distance D as it is
    not needed for smoothing logic

    theta_map = np.arctan2(u2_bar, u1_bar) % np.pi

    for i in range(1, q + 1):
        theta_i = (i - 1) * (np.pi / q)
        diff = np.abs(theta_map - theta_i)
        diff = np.minimum(diff, np.pi - diff)

        #Applies smoothing factor (cosine raised to power K)
        smooth_factor = np.cos(diff) ** K
        H[:, :, i] *= smooth_factor

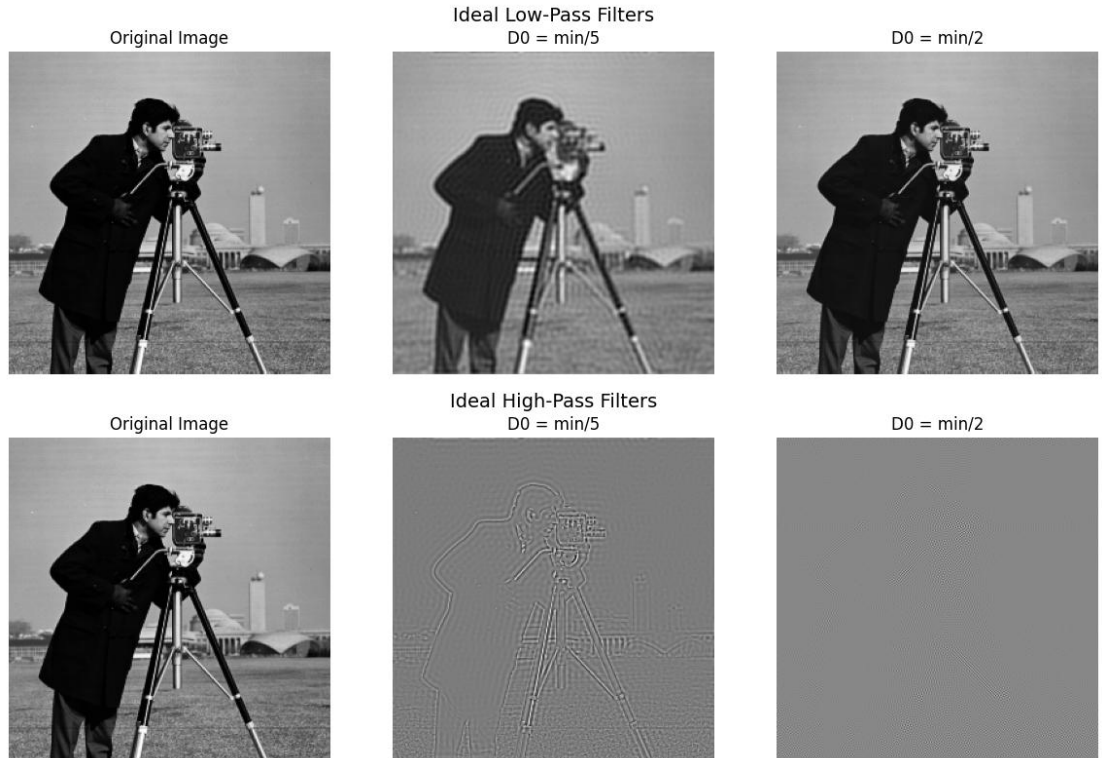
    return H
```

PART 3: Main Block-demo

Execution of demo.py & Results Commentary

The main script, demo.py, orchestrates the execution of the assignment requirements. Initially, the reference image cameraman.png is read, converted to grayscale using the pillow library, and cast to a floating-point array. Subsequently, the requested filter banks are constructed and applied, with the results being visualized via the matplotlib library.

1. Ideal Filters (Low-Pass and High-Pass)



Commentary: Ideal low-pass (H_{IL1} , H_{IL2}) and high-pass (H_{IH1} , H_{IH2}) filters were generated using cutoff frequencies $D_{01} = \min(N_1, N_2)/5$ and $D_{02} = \min(N_1, N_2)/2$. The ideal low-pass filters successfully blur the image by retaining only the low frequencies. However, due to the sharp step-function cutoff in the frequency domain, a severe "ringing effect" is produced in the spatial domain, which is highly visible around the high-contrast edges of the Cameraman. Conversely, the ideal high-pass filters zero out the central spectrum (DC and low frequencies), effectively preserving only the sharp edges and details. As the cutoff radius D_0 increases, the low-pass filter allows more spatial information to pass (resulting in less blur), while the high-pass filter retains less information (resulting in a darker image with only the sharpest edges visible).

2. Gaussian Filters

(Low-Pass and

High-Pass)

Original Image



Gaussian Low-Pass Filters
 $D_0 = \min/5$



$D_0 = \min/2$



Original Image



Gaussian High-Pass Filters
 $D_0 = \min/5$



$D_0 = \min/2$



Commentary: Repeating the experimental process with Gaussian filters for the identical cutoff frequencies yields noticeably different spatial results. Because the Gaussian function features a smooth transition without abrupt discontinuities, its corresponding inverse transform in the spatial domain is also smooth. Consequently, Gaussian low-pass filtering blurs the image efficiently without introducing any distracting ringing artifacts. Similarly, the Gaussian high-pass filters highlight the structural outlines of the image in a much more natural and visually pleasing manner compared to the ideal filters.

3. Directional

Filter

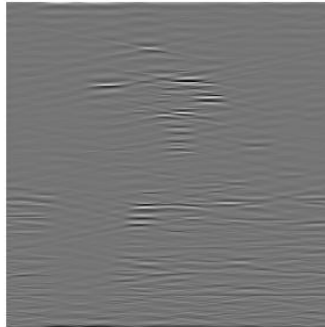
Bank

Ideal Directional Filter Bank ($q=8$)

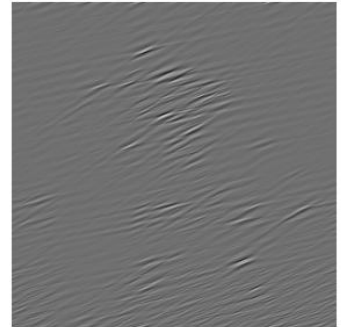
H0: Low-Pass



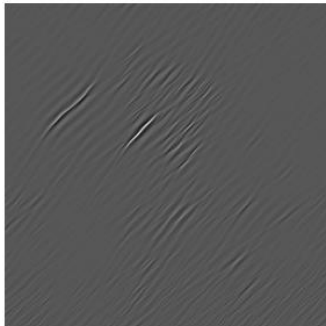
H1: Directional



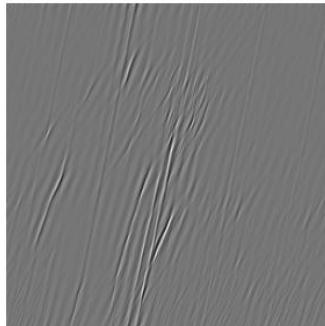
H2: Directional



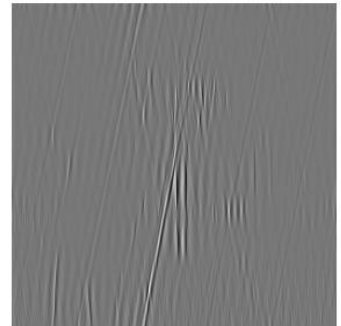
H3: Directional



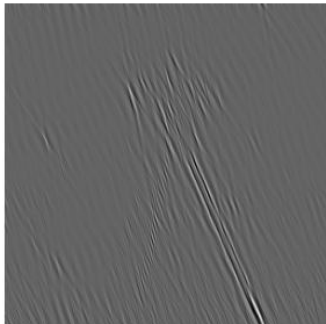
H4: Directional



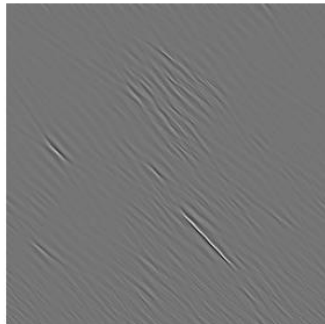
H5: Directional



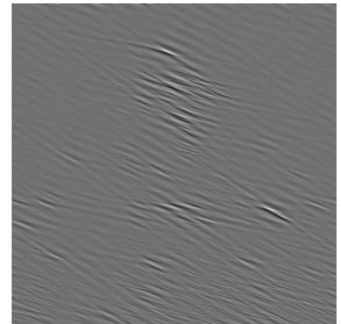
H6: Directional



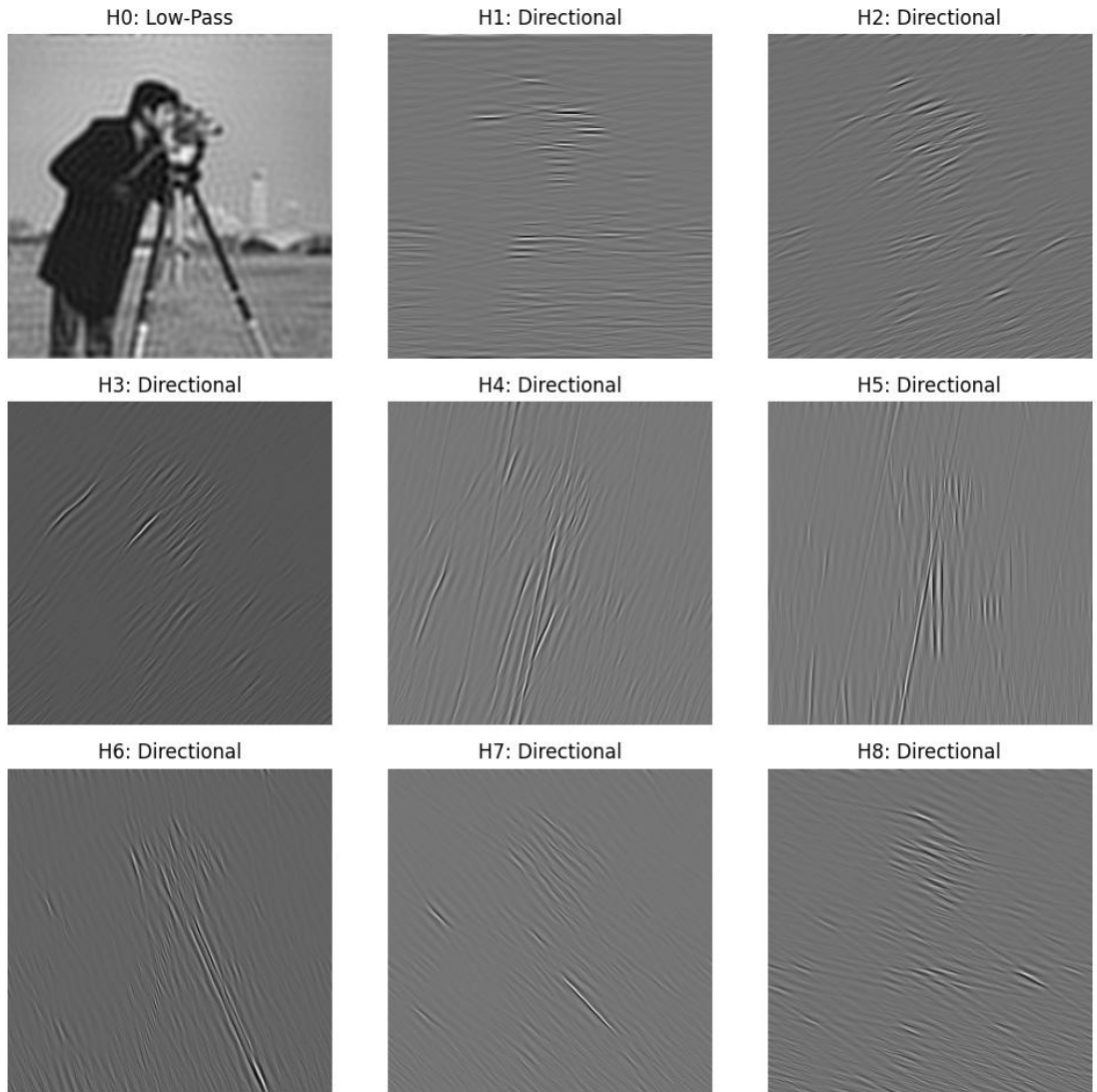
H7: Directional



H8: Directional



Smoothed Directional Bank ($q=8$, $K=2$)



Commentary: Two distinct filter banks were constructed utilizing $D_0 = \min(N_1, N_2)/8$ and $q=8$ directional slices: one utilizing ideal boundaries and one implementing smoothing (with hyperparameter $K=2$). In both arrays, the central filter H_0 outputs the heavily blurred background of the image. The subsequent filters (H_1 through H_8) successfully isolate waves propagating in specific angular ranges. In the spatial domain, a directional filter inherently enhances the edges that are **perpendicular** to the filter's frequency passband. For instance, frequency filters allowing horizontal waves to pass vividly reveal strong vertical spatial features (e.g., the background buildings and the vertical sections of the tripod). When comparing the ideal directional filters to the smoothed ones, it is evident that the introduction of the $\cos^K(\cdot)$ smoothing factor significantly suppressed optical artifacts (ringing) across the slice boundaries, delivering a much cleaner and well-defined edge separation.